

TURNIX SALUD

Sistema Integrado de Salud

Plataforma multi-rol (Paciente · Médico · Admin)
con turnos en tiempo real, recetas firmadas
digitalmente y arquitectura cloud-ready.

FastAPI

Spring Gateway

Supabase

Docker · Render

Defensa técnica · 15 min

Autor

Rodrigo García

Proyecto Fin de Ciclo

Repositorio

RodrigoGarcia2004 / Turnix-Project

Producción

turnix-project.onrender.com

turnix-gateway.onrender.com

Año

2026

Índice de la defensa

15 min · audiencia técnica · demo en vivo incluida

01 Problema y motivación

¿Por qué Turnix?

02 Visión y alcance

Roles, casos de uso, KPIs

03 Arquitectura del sistema

3 capas + gateway

04 Stack tecnológico

Decisiones y por qué

05 Modelo de datos

Esquema Supabase Postgres

06 Flujo: Paciente

Registro, turno, chat, recetas

07 Flujo: Médico

Consulta + receta firmada

08 Flujo: Admin

Gobierno de la plataforma

09 Tiempo real con WebSockets

Chat y notificaciones

10 Firma digital manuscrita

Canvas → PDF embedido

11 Gateway Java + Worker

Spring Cloud + métricas

12 Seguridad y RGPD

Re-auth, password, datos

13 Despliegue (Render + CI/CD)

Docker, render.yaml

14 Demo en vivo

Guion paso a paso

15 Roadmap y conclusiones

Lo aprendido

01 · Problema y motivación

Tres dolores reales del circuito asistencial digital

Salas de espera ciegas

El paciente llega sin saber su turno real; el médico no controla el flujo de pacientes en cola.

Recetas en papel

El médico imprime, firma a mano y entrega físicamente. El paciente puede perder la receta y no hay trazabilidad.

Sistemas desconectados

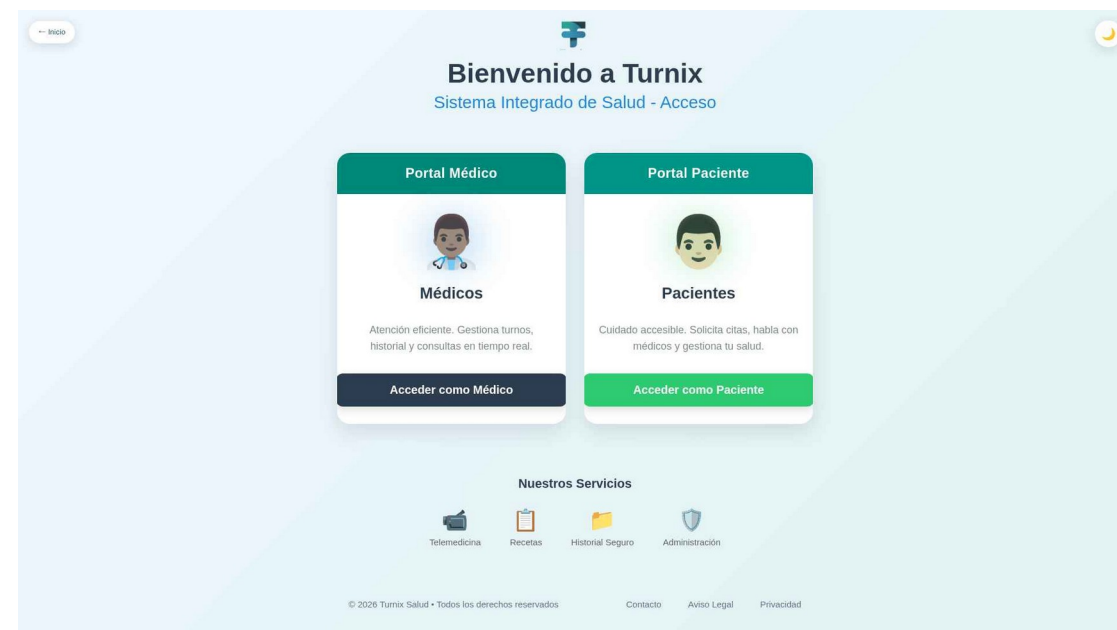
Agenda, chat, historial y prescripción son herramientas distintas → fricción para todos los roles.

Turnix unifica turno + consulta + receta digital en una sola plataforma.

02 · Visión y alcance

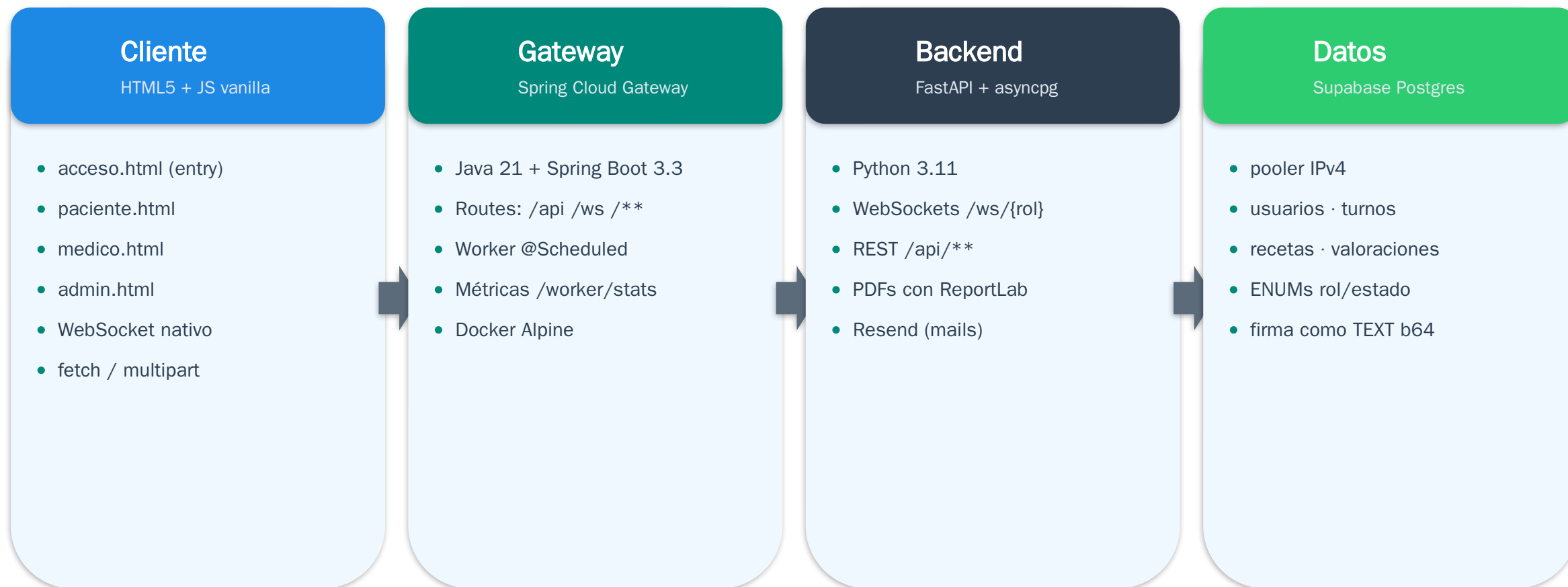
Una sola puerta de entrada · 3 perfiles · 1 base de datos

- Paciente: registro con verificación de email, solicitud de turno por especialidad, chat con el médico y descarga de recetas en PDF.
- Médico: panel con pacientes en espera, valoración del médico, generación de recetas con firma manuscrita digital.
- Admin: alta/edición/borrado de usuarios, gestión de solicitudes de cambio de especialidad, historial de conexiones.
- Tiempo real: turnos, chats y nuevas recetas se propagan por WebSocket sin recargar.
- Cloud-ready: despliegue automático en Render desde GitHub, gateway en contenedor Docker.



03 · Arquitectura del sistema

Cliente → Gateway Java → Backend FastAPI → Supabase (Postgres)



04 · Stack tecnológico y por qué

Frontend	HTML5 + CSS3 + JS vanilla	Cero build-step, tiempo de carga mínimo, sin lock-in de framework.
Backend	FastAPI 0.115 (Python 3.11) + asyncpg	API async, OpenAPI auto, ideal para WebSockets y endpoints REST.
PDF	ReportLab 4 + Pillow	Generación on-the-fly de la receta con la firma manuscrita embebida.
Gateway	Spring Cloud Gateway 2023.0.3 (Java 21)	Reactive, expresivo en routes, ecosistema enterprise para presentar.
BD	Supabase Postgres (pooler IPv4)	PostgreSQL gestionado, free-tier, backups automáticos.
Mensajería	WebSockets nativos + Resend para email	Chat y notificaciones sin polling; Resend cubre el doble opt-in.
CI/CD	Render + GitHub (auto-deploy desde rama)	Push a cambios → deploy automático del backend y del gateway.
Tests	Pytest async + httpx + Playwright (manual)	Cobertura sobre las rutas críticas: auth, recetas y PDF.

05 · Modelo de datos (Supabase Postgres)

usuarios

- id SERIAL PK
- usuario TEXT UNIQUE
- contraseña TEXT
- rol ENUM rol_t
- nombre_completo
- correo_electronico
- especialidad
- email_verificado BOOL
- foto_perfil BYTEA

turnos

- id SERIAL PK
- id_paciente → usuarios.id
- especialidad
- atendido_por → usuarios.id
- numero_turno
- estado ENUM estado_t
- fecha · fecha_fin
- puntuacion · valoracion

recetas

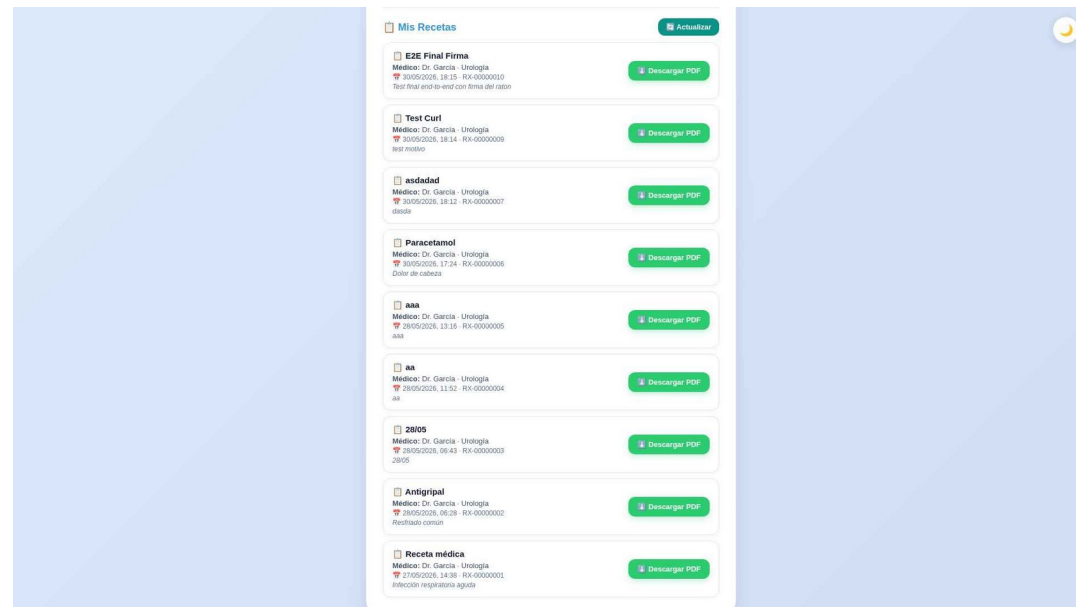
- id SERIAL PK
- paciente_id → usuarios.id
- medico_id → usuarios.id
- turno_id → turnos.id (UQ nullable)
- nombre_receta · motivo
- medicamento · dosis · duracion
- indicaciones
- firma_base64 TEXT (PNG dataURL)
- fecha_emision TIMESTAMP

ENUMs: rol_t (PACIENTE · MEDICO · ADMIN), estado_t (EN_ESPERA · EN_CONSULTA · COMPLETADO · CANCELADO)

Decisión clave: firma se almacena como dataURL PNG en TEXT, simple de embeber luego en ReportLab.

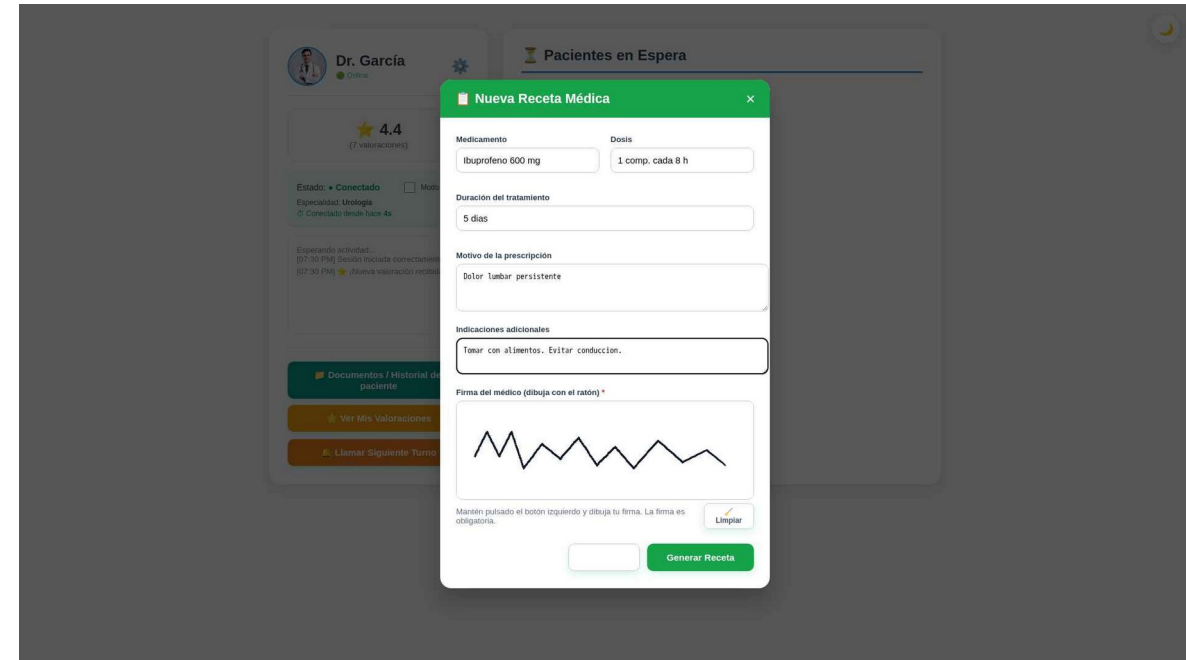
06 · Flujo: Paciente

- Registro con email (Resend envía código de verificación de 6 dígitos).
- Login WS: LOGIN:user:pass → backend devuelve LOGIN_OK:PACIENTE:nombre:id.
- Solicitud de turno: el paciente elige especialidad y entra en cola.
- Chat médico-paciente en tiempo real durante la consulta.
- Sección «Mis Recetas» (nuevo en esta entrega) → lista todas sus prescripciones con código RX-xxxxxxx.
- Descarga PDF: GET /api/recetas/{id}/pdf?user_id={uid} ⇒ PDF con firma embebida.
- Notificación push WS: RECETA_NUEVA:{nombre} refresca la lista al instante.



07 · Flujo: Médico

- Login dedicado (medico.html) con verificación de rol MEDICO en servidor.
- Panel con valoración 4.4/5 + lista de pacientes en espera en vivo.
- Al llamar a un turno: chat WS, acceso al historial del paciente, ver documentos.
- Generación de receta con modal:
 - Datos clínicos (medicamento, dosis, duración, motivo, indicaciones).
 - Canvas HTML5 para firmar con el ratón o trackpad (toda la lógica en JS vanilla).
 - Confirmación con re-autenticación de password del propio médico.
- El backend valida password, persiste la receta en Postgres y notifica al paciente por WS.



08 · Flujo: Admin (modo claro + oscuro)

Tumix · Administración

Conectado como admin [Cerrar ses](#)

Usuarios Solicitudes de especialidad Historial de conexiones

Lista de usuarios [+ Refrescar](#)

ID	USUARIO	NOMBRE COMPLETO	EMAIL	ROL	ESPECIALIDAD	VERIFICADO	ACCIÓN
1	medico1	Dr. Garcia	garciaherediaarodrig@gmail.com	medico	Urología	✓	Editar Borrar
2	medico2	Dra. Ana Martinez	garciaherediaarodrig@gmail.com	medico	Medicina General	✓	Editar Borrar
3	admin	Administrador	garciaherediaarodrig@gmail.com	ADMIN	—	✓	Editar Borrar usuario protegido
4	user1	Juan Pérez	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar
7	TEST_6043236f	TEST Updated Name	garciaherediaarodrig@gmail.com	medico	Medicina General	✓	Editar Borrar
8	TEST_8fae740d	TEST Outsider	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar
9	TEST_37913046	TEST Updated Name	garciaherediaarodrig@gmail.com	medico	Medicina General	✓	Editar Borrar
10	TEST_117b18b7	TEST Outsider	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar
21	nrtn	nrtn nrtn	garciaherediaarodrig@gmail.com	usuario	—	⚠	Editar Borrar
22	PruebaLlegada	PruebaLlegada PruebaLlegada	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar

Tema claro

Tumix · Administración

Conectado como admin [Cerrar ses](#)

Usuarios Solicitudes de especialidad Historial de conexiones

Lista de usuarios [+ Refrescar](#)

ID	USUARIO	NOMBRE COMPLETO	EMAIL	ROL	ESPECIALIDAD	VERIFICADO	ACCIÓN
1	medico1	Dr. Garcia	garciaherediaarodrig@gmail.com	medico	Urología	✓	Editar Borrar
2	medico2	Dra. Ana Martinez	garciaherediaarodrig@gmail.com	medico	Medicina General	✓	Editar Borrar
3	admin	Administrador	garciaherediaarodrig@gmail.com	ADMIN	—	✓	Editar Borrar usuario protegido
4	user1	Juan Pérez	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar
7	TEST_6043236f	TEST Updated Name	garciaherediaarodrig@gmail.com	medico	Medicina General	✓	Editar Borrar
8	TEST_8fae740d	TEST Outsider	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar
9	TEST_37913046	TEST Updated Name	garciaherediaarodrig@gmail.com	medico	Medicina General	✓	Editar Borrar
10	TEST_117b18b7	TEST Outsider	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar
21	nrtn	nrtn nrtn	garciaherediaarodrig@gmail.com	usuario	—	⚠	Editar Borrar
22	PruebaLlegada	PruebaLlegada PruebaLlegada	garciaherediaarodrig@gmail.com	usuario	—	✓	Editar Borrar

Tema oscuro

- Lista de usuarios con badge de rol, especialidad y verificación de email.
- Acciones críticas (editar/borrar/aprobar especialidad) requieren reautenticación con la password del admin.
- Toggle de tema persiste en localStorage; las variables CSS se intercambian sin recargar.

09 · Tiempo real con WebSockets

FastAPI gestiona 3 endpoints WS, uno por rol, con su propio pool de conexiones.

Endpoints WS

`/ws/PACIENTE`

`/ws/MEDICO`

`/ws/ADMIN`

Vocabulario de mensajes

`LOGIN_OK:rol:nombre:id`

Sesión establecida

`NEW_TURN0:numero:user`

Médico ve nuevo paciente

`CHAT:remitente:mensaje`

Chat 1-a-1

`RECETA_NUEVA:nombre`

Push al paciente

`CONFIRMAR_ASISTENCIA`

Paciente confirma turno

`ESPECIALIDAD_OK / KO`

Admin notifica decisión

10 · Firma digital manuscrita

Del canvas del navegador al PDF en la BD

- 1

Canvas HTML5

El médico dibuja con mouse/touch. Listeners mousedown/move/up + touchstart/move/end.
- 2

toDataURL('image/png')

Se exporta como PNG codificado en base64 (``data:image/png;base64,...``).
- 3

Validación frontend

Si `firmaTieneTrazo == false` → no se permite enviar. Mensaje rojo en el modal.
- 4

POST /api/recetas/crear

Body JSON incluye `firma_base64`. El backend re-autentica al médico antes de insertar.
- 5

ReportLab + Pillow

Al pedir el PDF: `PIL.verify()` defensivo y `RLImage` embebe la firma a 7 × 3.2 cm.



Datos del paciente	
Nombre completo	Juan Pérez
Usuario	user1
Correo electrónico	garciaherediarodrigo@gmail.com

Detalles de la receta	
Nombre de la receta	E2E Final Firma
Motivo	Test final end-to-end con firma del raton
Medicamento	E2E Final Firma
Dosis	1 cada 8h
Duración del tratamiento	7 días
Fecha de emisión	30/05/2026 18:15

Profesional sanitario	
Médico	Dr. García
Especialidad / Rama	Urología
Usuario interno	medico1
Correo profesional	garciaherediarodrigo@gmail.com

Firma del médico:

11 · Gateway Java + Worker programado

Spring Cloud Gateway · Java 21 · Docker Alpine

- Routes declarativas en application.yml: /api/** → backend, /ws/** → backend WS, /** → frontend estático.
- RequestCountingFilter (GlobalFilter) cuenta peticiones por ruta y status code.
- Worker @Scheduled cada 15 s ⇒ ping a /api/health del backend, almacena latencia y estado.
- Endpoint /worker/stats expone JSON con uptime, rutas más usadas, p50/p95 y salud del backend.
- Header X-Powered-By: Turnix-Gateway añadido globalmente para trazabilidad.

GET /worker/stats → 200 OK

```
{
  "uptime_seconds": 1432,
  "total_requests": 318,
  "requests_by_route": { "/api": 217, "/ws": 84, "other": 17 },
  "responses_by_status_class": { "2xx": 295, "4xx": 18, "5xx": 5 },
  "backend": {
    "status": "UP",
    "last_latency_ms": 64,
    "checks_ok": 96,
    "checks_fail": 1
  }
}
```

Componentes Java

- **HealthCheckWorker**
@Scheduled · pinguea /api/health
- **RequestCountingFilter**
GlobalFilter · cuenta tráfico
- **MetricsStore**
ConcurrentHashMap + LongAdder
- **WorkerStatsController**
GET /worker/stats → JSON

12 · Seguridad y privacidad

Reautenticación en acciones críticas

Cualquier acción admin (alta/baja/edición) y la firma de receta exigen volver a introducir la password.

Comprobación de rol en servidor

Cada endpoint REST verifica el rol del usuario (PACIENTE/MEDICO/ADMIN) → un paciente no puede llamar /api/admin/**.

Aislamiento de recetas

GET /api/recetas/{id}/pdf rechaza con 403 si el user_id no es el paciente dueño ni el médico que la emitió.

CORS controlado en gateway

El gateway aplica una política única; el backend confía en él. Reduce superficie y centraliza configuración.

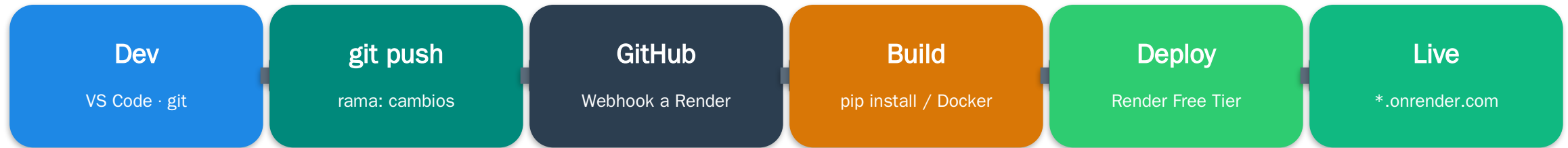
Datos sensibles

Email verificado vía Resend con código de 6 dígitos antes de poder operar (cumple RGPD para datos médicos).

Limpieza de logs

No se loguean contraseñas; los mensajes de auth fallida sólo dicen «credenciales inválidas».

13 · Despliegue continuo (Render + GitHub)



Servicio 1 · Backend FastAPI

- Runtime: Python 3.11.9
- Build: `pip install -r requirements.txt`
- Start: `uvicorn server:app --host 0.0.0.0 --port $PORT`
- Envs: `SUPABASE_*`, `RESEND_API_KEY`, `FRONTEND_DIR`, `UPLOADS_DIR`, `CORS_ORIGINS`
- URL: `turnix-project.onrender.com`

Servicio 2 · Gateway Java (Docker)

- Runtime: Eclipse Temurin 21 JRE Alpine
- Build: Dockerfile multistage (Maven 3.9 + JRE)
- rootDir: `turnix-gateway`
- Envs: `BACKEND_URL`, `BACKEND_URL_WS`
- URL: `turnix-gateway.onrender.com`

14 · DEMO EN VIVO

Guion para los próximos 4 minutos en pantalla

00:00

Acceso

Entrar en turnix-project.onrender.com → redirige a [acceso.html](#).

00:30

Paciente

Login user1 / 1234. Mostrar foto de perfil + sección Recetas (lista poblada).

01:15

Médico

Abrir otra pestaña [medico.html](#). Login medico1 / 1234. Llamar siguiente turno.

02:00

Receta + firma

Modal de receta, rellenar campos, dibujar firma con el ratón, generar.

02:45

Vuelta al paciente

Push WebSocket → la receta aparece sola. Descargar PDF y mostrarlo.

03:20

Admin

Login admin / 1234. Cambiar de modo claro a oscuro. Mostrar tabla de usuarios.

03:45

Gateway

Abrir turnix-gateway.onrender.com/worker/stats en una pestaña. JSON en vivo.

Decisiones técnicas más relevantes

► ¿Por qué FastAPI y no Flask/Django?

Necesitamos WebSockets nativos, async I/O y OpenAPI gratis. FastAPI lo da out of the box.

► ¿Por qué frontend en HTML puro y no React?

El alcance lo permite, el time-to-render es mínimo y no necesitamos build pipeline. Más fácil de defender ante un tribunal.

► ¿Por qué un gateway en Java?

Suma valor académico (segundo lenguaje, contenedor Docker, microservicio real) y permite añadir métricas y políticas sin tocar el backend.

► ¿Por qué firma como dataURL en TEXT?

Evita gestionar un object storage extra; cabe sin problemas para una imagen vectorial de ~10 KB.

► ¿Por qué Supabase en vez de Postgres autogestionado?

Free-tier real con backups, IPv4 pooler, y el código sigue siendo Postgres estándar (asyncpgr).

► ¿Por qué WebSocket y no SSE?

El chat es bidireccional. SSE serviría solo para push del backend, no para el paciente enviando.

Roadmap y lo que aprendí

Hecho

- Turnos en tiempo real y chat
- Recetas con firma manuscrita + PDF
- Panel admin con modo claro/oscuro
- Email verificado por código
- Gateway Java + worker + métricas
- CI/CD a Render

Siguiente

- QR de verificación en cada PDF
- Notificaciones push reales (web push)
- Multi-clínica (tenants)
- Calendario de turnos al estilo Google
- Histórico exportable a CSV
- Tests E2E con Playwright en CI

Futuro

- Videollamada nativa (WebRTC)
- Receta con QR firmada digitalmente
- App móvil React Native
- IA: pre-diagnóstico por síntomas
- Integración con receta electrónica oficial
- Estadísticas tipo PowerBI

Gracias

Preguntas, código y demo en vivo a disposición del tribunal

github.com/RodrigoGarcia2004/Turnix-Project · turnix-project.onrender.com · turnix-gateway.onrender.com

Rodrigo García · 2026